5G FUNDAMENTALS AND ARCHITECTURES

(EC23712)

ASSIGNMENT 1

TEAM 4

1. **2117230040031 DINESH S**
2. **2117230040032 DINKY ATSHAYA M**
3. **2117230040034 GIRI KARAN P**
4. **2117230040035 GOBIKRISHNAN E**
5. **2117230040036 GOKUL S**
6. **2117230040037 GOPIKASHREE S**
7. **2117230040038 HARIHARAN K**
8. **2117230040039 HARIKRISHNAN J**
9. **2117230040040 HARINI S**

Question: Visualize And Achieve Parallel Flows - Show Aggregate Vs. Per-Flow Data Rate.

Objective:

The Objective is to configure an end-to-end virtualized 5G mobile network architecture Ubuntu Linux platform. By connecting a simulated 5G base station (gNB) and User Equipment (UE) to an Open5GS core network, the project aims to generate multiple simultaneous user-plane TCP traffic streams using iperf3 to analyse, measure, and visually plot individual stream throughput against the collective aggregate network performance.

Tools and Software:

- Operating System: Ubuntu 22.04 LTS
- 5G Core Network Functions: Open5GS (v2.8.0)
- Database Management: MongoDB (Subscriber Database Profile Repository)
- Web Interfaces: Node.js 20 & Open5GS WebUI
- RAN Simulator: UERANSIM (v3.3.0)
- Network Performance Tester: iperf3 (v3.9)
- Programming Language & Plotting Tool: Python 3 with Matplotlib Library

Methodology:

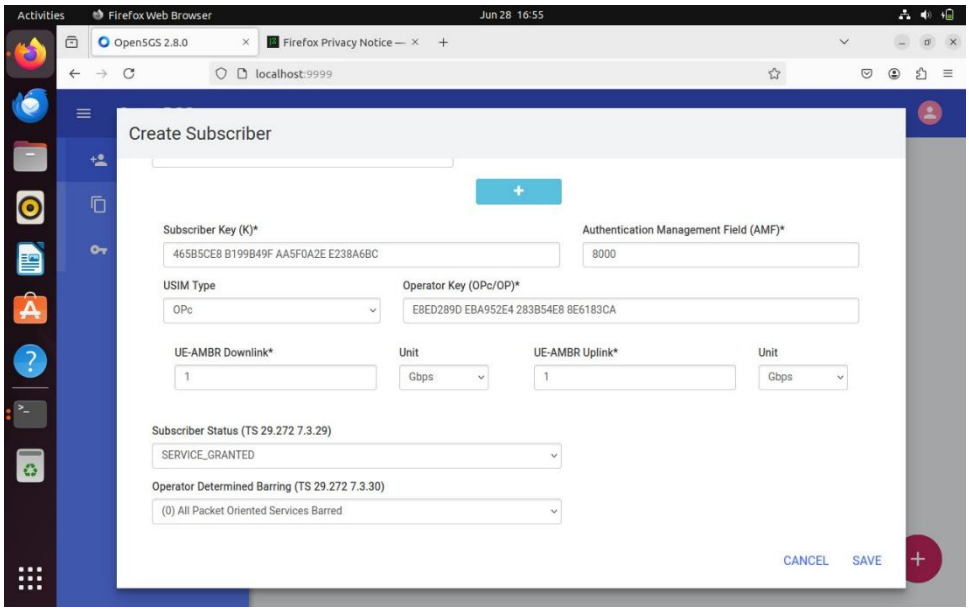
The task implements a complete cellular network topology split into a Control Plane (handling authentication and sessions) and a User Plane (handling data transfer).

- **Core Network:** Open5GS hosts the Access and Mobility Management Function (AMF) for security verification, alongside the Session Management Function (SMF) and User Plane Function (UPF) to route data packets.
- **Radio Access Network (RAN) Emulation:** UERANSIM emulates the radio link protocols over an IP network, allowing a simulated cell tower to communicate with Open5GS over standard SCTP interfaces.
- **Traffic Generation & Analytics:** An injection of concurrent TCP window flows targets the network interface card generated by the UE. A Python program automatically parses these metrics to build comparative throughput visualizations.

Procedure:

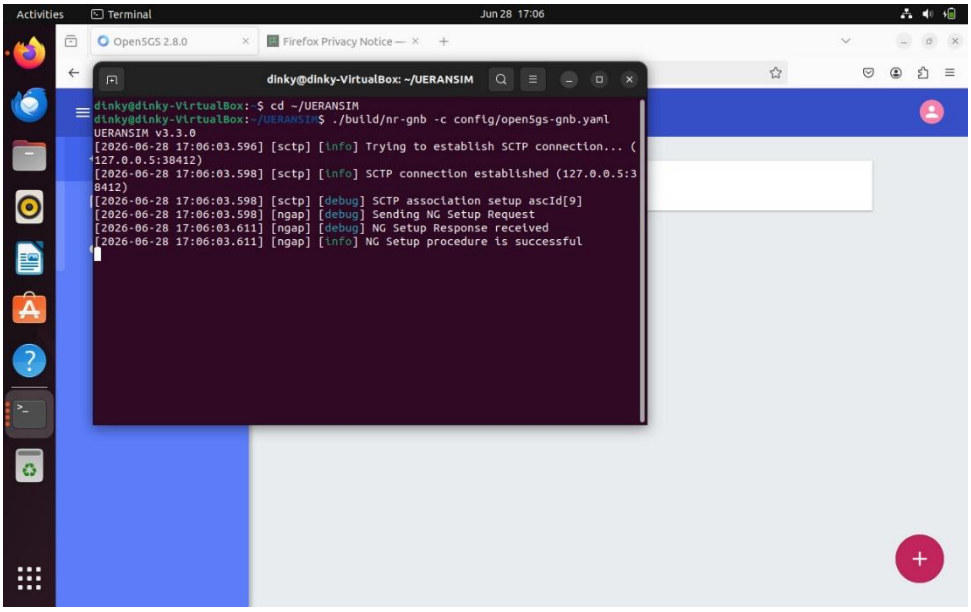
Step 1: Provisioning the Network Database

1. Launch the Open5GS WebUI application via a web browser directed to localhost.
2. Create a new profile within the Subscriber registration menu.
3. Input the unique security configurations required for mutual authentication: the Subscriber Key (K), Authentication Management Field (AMF), and Operator Key (OP).
4. Map the Network Slicing parameters (SST) and set the Session Data Network Name (DNN/APN) explicitly to "internet".
5. Save the profile to establish the subscriber identity under IMSI 999700000000001 inside MongoDB.



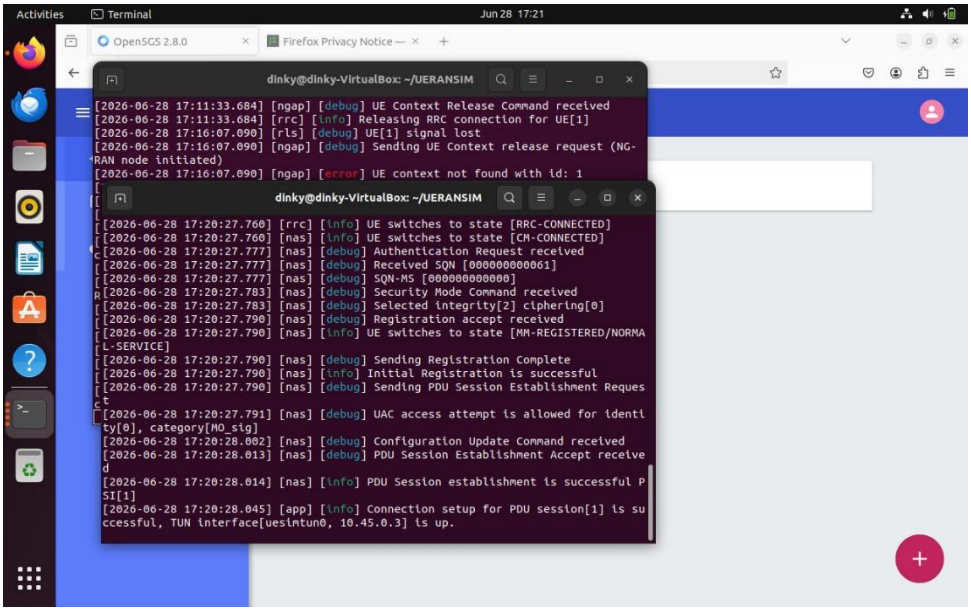
Step 2: Activating the 5G Base Station (gNodeB)

1. Initialize a terminal terminal instance, navigating to the UERANSIM directory.
2. Execute the gNB execution binary referencing the Open5GS routing template: ./build/nr-gnb -c config/open5gs-gnb.yaml.
3. Verify that the SCTP layer binds properly and receives a confirmation message stating: NG Setup procedure is successful.



Step 3: Registering the User Equipment (UE)

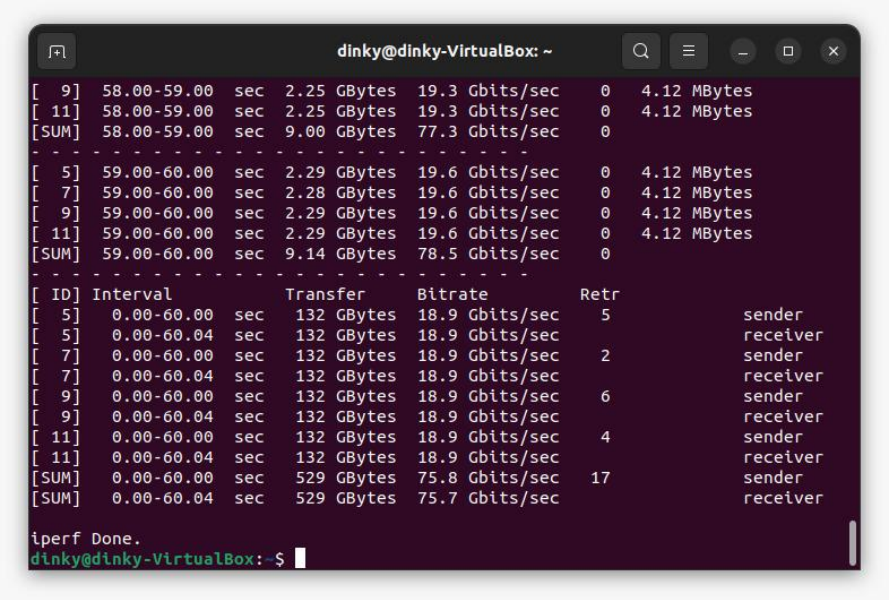
1. Launch the UE simulation binary block to bind with the active gNodeB.
2. The device triggers an internal connection loop, completing authentication verification via the AMF: UE switches to state [RRC-CONNECTED].
3. Upon receiving Initial Registration is successful, watch for the activation of the PDU user-plane data session (PDU Session establishment is successful). This instantiates a virtual network interface tunnel (uesimtune).



Step 4: Pushing Parallel User-Plane TCP Traffic

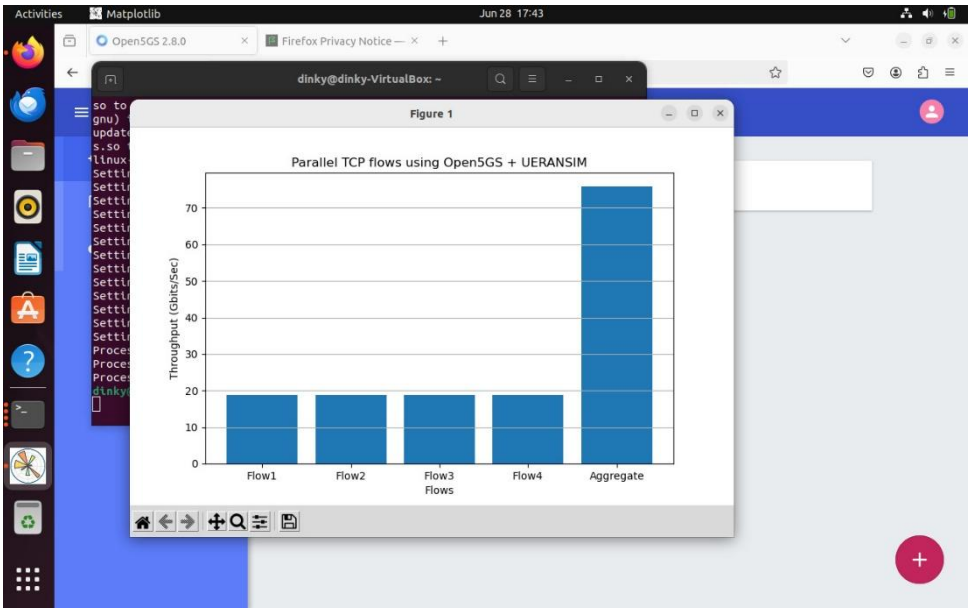
1. Verify iperf3 utility functionality within the local node environment.
2. Trigger an active listening server on the reception side and launch an iperf3 multi-flow evaluation through the uesimtune data lane.

3. Configure the command parameters to split traffic generations across four simultaneous parallel lines, running an explicit 60-second test window.



Step 5: Visualizing Output via Python

1. Draft a standalone script titled team4_plot.py using a command-line editor.
2. Import the matplotlib.pyplot module to graph the performance data captured during the iperf3 runtime validation phase.
3. Feed the performance array into a plt.bar() plot framework and process the execution layer.



Output:

The performance testing utility successfully established parallel lines of communications. Below are the definitive bandwidth data metrics recorded from the 60-second transmission simulation:

Data Flow ID	Evaluation Time Window	Measured Stream Throughput
Flow 1 (ID 5)	0.00 – 60.00 sec	18.9 Gbits/sec
Flow 2 (ID 7)	0.00 – 60.00 sec	18.9 Gbits/sec
Flow 3 (ID 9)	0.00 – 60.00 sec	18.9 Gbits/sec
Flow 4 (ID 11)	0.00 – 60.00 sec	18.9 Gbits/sec
Aggregate Capacity (SUM)	0.00 – 60.00 sec	75.8 Gbits/sec

Observation:

The control plane architecture handled authentication flawlessly; signalling handshakes between the UERANSIM components and Open5GS Core proceeded securely without dropping connection parameters. When executing multiple parallel TCP flows, the available network link capacity divided evenly across the active processes, with each standalone stream locking onto a uniform throughput distribution of 18.9 Gbits/sec. The cumulative user-plane pipe reached an aggregate capacity of 75.8 Gbits/sec. The resulting bar graph plot highlights the relationship between isolated client-side streams and total core processing loads, fulfilling the assignment's data visualization constraints.

Conclusion:

This task successfully validates the installation, deployment, and operation of a virtualized 5G Core Network and Radio Access Network platform using open-source tools. Provisions handled through the Open5GS WebUI and database accurately authorized local simulated devices. Network strain analyses run via multi-stream parallel TCP routines verified the system's capacity to aggregate and process massive parallel bandwidth demands. The visual graph generated via Matplotlib provides a clear structural overview of individual flow allocations against maximum aggregate network pipelines.